Code documentation

outside of the code

1. Write new documentation outside of the code (or write the documentation when coding).
    - Often markdown
2. Generate documentation from docstrings in the code.
    - Often HTML

1. Markdown

What is markdown?

Markdown is a lightweight markup language. The main point with Markdown is that it provides some structure to the text while it is also:

- **Easy to read in a normal text editor.**

It is, for instance, used for:

- README.md files, e.g. on GitHub
- As the markup language to use for describing your work in your Jupyter Notebook, that is not part of the code.

By JOHN GRUBER

Markdown

Main | Basics | Syntax | License | Dingus

ARCHIVE
THE TALK SHOW
DITHERING
PROJECTS
CONTACT
COLOPHON
FEEDS / SOCIAL
SPONSORSHIP

DOWNLOAD

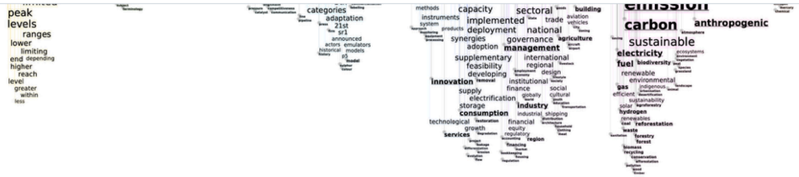
Markdown 1.0.1 (18 KB) — 17 Dec 2004

INTRODUCTION

Markdown is a text-to-HTML conversion tool for web writers. Markdown allows you to write using an easy-to-read, easy-to-write plain text format, then convert it to structurally valid HTML.

https://daringfireball.net/projects/markdown/basics

README BSD-2-Clause license



Example of word rains for IPCC reports.

Running

See `visualise_climate_texts.py` for an example of how the code in `wordrain/generate_wordrain.py` is to be used.

You need a word2vec space, a stop word list and a corpus containing several sub-corpora. The folder CORPUS_FOLDER should contain sub-folders, which each of them contain the sub-corpora to be visualised. The sub-folders, in turn, contain (one or several) .txt-files, where the corpus is stored. The folder `swedish-climate-texts` provides an example of this structure. The Swedish word2vec space used in the example (model-sv.bin) can be found here: http://vectors.nlpl.eu/repository/

```
WORD_SPACE_PATH = "./model-sv.bin"
STOP_WORDS_LIST = "swedish_stopwords_climate.txt"
CORPUS_FOLDER = "swedish-climate-texts"
OUTPUT_FOLDER = "images_climate"
```

Web service

There is a web service, where word rains can be produced from uploaded texts: wordrain.org

Contributors and funding

Word Rain is a collaboration between the Centre for Digital Humanities and Social Sciences, Uppsala (CDHU), the

Example of README.md on GitHub

```
3
4  ![Example of word rains](word-rain-examples/unified-english-images-climate.png)
5  <em>Example of word rains for IPCC reports.</em>
6
7  ## Running
8  See `visualise_climate_texts.py` for an example of how the code in
       `wordrain/generate_wordrain.py` is to be used.
9
10 You need a word2vec space, a stop word list and a corpus containing several sub-corpora. The
       folder CORPUS_FOLDER should contain sub-folders, which each of them contain the sub-corpora
       to be visualised. The sub-folders, in turn, contain (one or several) .txt-files, where the
       corpus is stored. The folder ``swedish-climate-texts`` provides an example of this
       structure. The Swedish word2vec space used in the example (model-sv.bin) can be found here:
       http://vectors.nlpl.eu/repository/
11
12 ```
13 WORD_SPACE_PATH = "./model-sv.bin"
14 STOP_WORDS_LIST = "swedish_stopwords_climate.txt"
15 CORPUS_FOLDER = "swedish-climate-texts"
16 OUTPUT_FOLDER = "images_climate"
17 ```
18 ## Web service
19 There is a web service, where word rains can be produced from uploaded texts:
20 [wordrain.org](https://wordrain.org)
21
```

Markdown for the README.md

Notebook Input Output Logs Comments (0) Settings

version" in the top-right corner.

Task a:

Copy this notebook by clicking the button 'Edit & Copy' in the top-right corner, so that you have your own version to work with. Save this version by clicking the button 'Save Version'.

Part I: Kaggle

In [1]:

```
from datetime import datetime # Import datetime from the datetime module in the Python Stan
dard Library
from datetime import timedelta # Import timedelta from the datetime module

current_time = datetime.now() # Get current date and time
print("Current date and time: " + str(current_time)) # Print current data and time. The dat
etime needs to be explicitly type converted into a string
```

Jupyter Notebook

```
In addition to the main aim of version control, you will also practice some Kaggle, the
[Markdown syntax](https://daringfireball.net/projects/markdown/basics), as well as get a bit
used to the notion of Python libraries.

Remember that when editing you Kaggle notebook, the changes are **not** automatically saved.
You need to press "Save version" in the top-right corner.
```

+ Code + Markdown

```
### Task a:
Copy this notebook by clicking the button 'Edit & Copy' in the top-right corner, so that you
have your own version to work with. Save this version by clicking the button 'Save Version'.
```

```
Part I: Kaggle
---------------
```

```
from datetime import datetime # Import datetime from the datetime module in the Python Standard Libra
from datetime import timedelta # Import timedelta from the datetime module
```

Markdown for the Jupyter Notebook

README.md for Word Rain

Here:

The following is used here:

- ## for title
- ### for a bit smaller title
- []() for link

(And # for an even larger title)

```
## Word Rain
Code for generating word rains (= a semantically
aware version of the word cloud visualisation of
texts)

Word Rain is a collaboration between [Centre for
Digital Humantieis and Social Sciences, Uppsala
(CDHU)](https://www.abm.uu.se/cdhu-eng/), the
[National Language Bank of Sweden/CLARIN Knowledge
Centre for the Languages of Sweden](https://
www.isof.se/other-languages/english/clarin-knowledge-
centre-for-the-languages-of-sweden-swelang) at the
Language Council of Sweden and the [iVis group]
(https://ivis.itn.liu.se) at Linköping University.

Example of a word rain of the IPCC report: 'Climate
Change 2021: The Physical Science Basis, Summary for
Policymakers'.
![Example of a word rain](word-rain-example.png)


### Installations
For a conda environment, the following is needed

conda install numpy
```

readme for topic-timelines (another CDHU repo)

Another syntax for the two levels of main titles is used

I.e, you can also:

- use ==== for highest level title,
- and — — for subtitle

```
topic-timelines
===============
This timeline visualises the prevalence of
topics over time. It uses the output of the
Topics2Themes tool, which automatically
extracts topics from a text corpus.

To run the code, you therefore first need to
run Topics2Themes on a text collection, and
then run the code here on the output from this
tool. You find Topics2Themes at: [https://
github.com/sprakradet](https://github.com/
sprakradet)

An example of how to run the code (and what is
needed for configuring the Topics2Themes tool)
is given in `plot_climate_timeline.py`.

Dependencies
------------
The code uses `numpy` and `matplotlib`.
```

README.md for Word Rain

For short code:

`visualise_climate_texts.py`

For longer code block:

```
WORD_SPACE_PATH = "../../../wordspaces/69/model.bin"
STOP_WORDS_LIST = "swedish_stopwords_climate.txt"
CORPUS_FOLDER = "swedish-climate-texts"
OUTPUT_FOLDER = "images_climate"
```

```
### Running
See `visualise_climate_texts.py` for an example of how the code in `wordrain/generate_wordrain.py` is to be used.

You need a word2vec space, a stop word list and a corpus containing several sub-corpora. The folder CORPUS_FOLDER should contain sub-folders, which each of them contain the sub-corpora to be visualised. The sub-folders, in turn, contain (one or several) .txt-files, where the corpus is stored. The folder "swedish-climate-texts", gives an example of this structure. The Swedish word2vec space used in the example can be found here: [http://vectors.nlpl.eu/repository/](http://vectors.nlpl.eu/repository/)

```

```
WORD_SPACE_PATH = "../../../wordspaces/69/model.bin"
STOP_WORDS_LIST = "swedish_stopwords_climate.txt"
CORPUS_FOLDER = "swedish-climate-texts"
OUTPUT_FOLDER = "images_climate"
```

README.md for Word Rain

- or * is used for lists

You can also use numbers instead of - (for numbered lists)

```
## Acknowledgements
The work on Word Rain is partly
supported by three research
infrastructures:

- [Huminfra](https://www.huminfra.se):
National infrastructure for Research in
the Humanities and Social Sciences
(Swedish Research Council, 2021-00176)
- [InfraVis](https://infravis.se): the
Swedish National Research
Infrastructure for Data Visualization
(Swedish Research Council, 2021-00181)
- [Nationella Språkbanken](https://
www.språkbanken.se/
sprakbankeninenglish.html): The
National Language Bank of Sweden
(Swedish Research Council, 2017-00626)
```

Read more about markdown

https://daringfireball.net/projects/markdown/basics

2. Documentation from docstrings

```
"""
Classes for practicing documentation
"""

class MyClass:
    """Class for practicing documentations

    Attributes
    ----------
    add_extra: int
    Always add this to sum

    """

    def __init__(self, add_extra):
        """
        Parameters
        ----------
        add_extra: int
        Always add this to sum
        """

        self.add_extra = add_extra
```

Simple example class

```
    def my_method(self, first_attr, second_attr):
        """Sum first_attr and second_attr and add_extra.

        Parameters
        ----------
        first_attr : int
        First part to sum

        second_attr : int
        Second part to sum


        Returns
        -------
        sum: int
        A sum of first_attr and second_attr and add_extra
        """

        return first_attr + second_attr + self.add_extra
```

Simple example class, continued

Generate pydoc with:

```
pydoc -w my_class
```

Note that you do **not** write my_class**.py** here

Or as follows (on Triton):

```
python3 -m pydoc -w
```

Class for practicing documentations

Attributes
---
add_extra: int
Always add this to sum

Methods defined here:

__init__(self, add_extra)
Parameters
---
add_extra: int
Always add this to sum

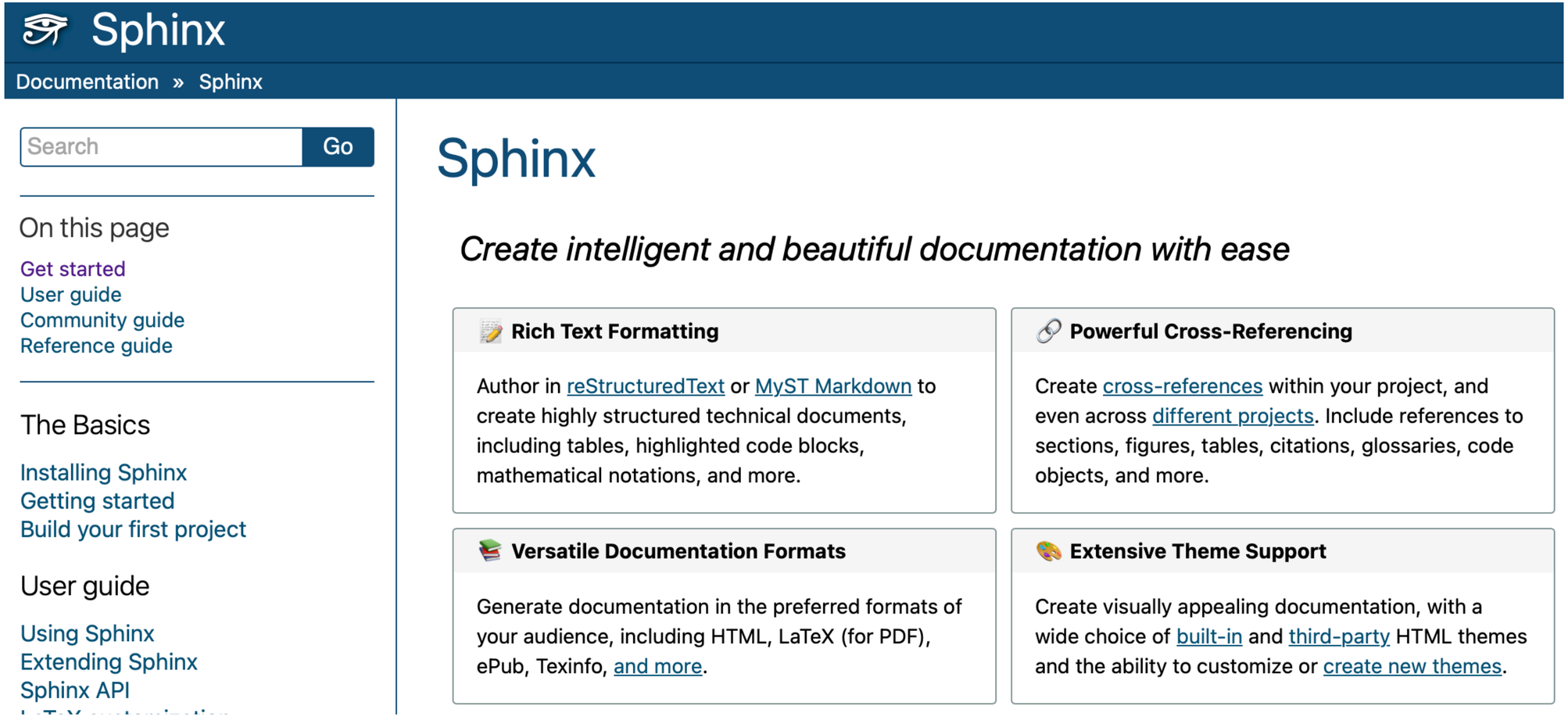
my_method(self, first_attr, second_attr)
Sum first_attr and second_attr and add_extra.

Parameters
---
first_attr : int
First part to sum

second_attr : int
Second part to sum

Returns
---
sum: int
A sum of first_attr and second_attr and add_extra

Resulting HTML



Often, more advanced tools than pydoc are used: E.g. Sphinx.

E.g. by scikit-learn

Ridge

class **sklearn.linear_model.Ridge(*alpha*=*1.0*, *, *fit_intercept*=*True*, *copy_X*=*True*, *max_iter*=*None*, *tol*=*0.0001*, *solver*='*auto*', *positive*=*False*, *random_state*=*None*)** [source]

Linear least squares with l2 regularization.

Minimizes the objective function:

```
||y - Xw||^2_2 + alpha * ||w||^2_2
```

This model solves a regression model where the loss function is the linear least squares function and regularization is given by the l2-norm. Also known as Ridge Regression or Tikhonov regularization. This estimator has built-in support for multi-variate regression (i.e., when y is a 2d-array of shape (n_samples, n_targets)).

Read more in the User Guide.

scikit-learn, a good inspiration for how to docstrings

```
def fit(self, X, y, sample_weight=None):
    """Fit Ridge regression model.

    Parameters
    ----------
    X : {ndarray, sparse matrix} of shape (n_samples, n_features)
        Training data.

    y : ndarray of shape (n_samples,) or (n_samples, n_targets)
        Target values.

    sample_weight : float or ndarray of shape (n_samples,), default=None
        Individual weights for each sample. If given a float, every sample
        will have the same weight.

    Returns
    -------
    self : object
        Fitted estimator.
    """
```